Iparhotel

Gestión de reservas de un hotel rural
Memoria del proyecto y análisis y diseño

Módulo MF0227

Alumno: Franco Irael Díaz Rodríguez

Noviembre de 2026

Índice

1. Introducción
2. Análisis: historias de usuario
3. Diseño: diagrama de casos de uso y diagrama de clases
4. Fase 1: modelo orientado a objetos
5. Fase 2: aplicación con Spring Boot
6. Fase 3: API REST con JWT y Angular
7. Pruebas
8. Cómo arrancar el proyecto
9. Conclusiones

1. Introducción

El hotel rural Iparhotel llevaba las reservas de sus habitaciones en una hoja de cálculo y quería pasar a una aplicación. En este proyecto he hecho tres cosas, una por cada fase: primero el modelo de clases en Java, después una web de gestión con Spring Boot y por último una API REST protegida con JWT y una pequeña aplicación en Angular para que los clientes puedan reservar.

Cada fase se apoya en la anterior. Las clases de la fase 1 son las mismas que luego se guardan en la base de datos en la fase 2, y el servicio de reservas de la fase 2 es el que usa la API de la fase 3. Todo está en un mismo repositorio de GitHub con dos carpetas: `iparhotel` (el proyecto Java con Maven) y `iparhotel-web` (el proyecto Angular).

Para empezar me basé en un proyecto que ya había hecho antes (IparGym, reservas de clases de un gimnasio), porque la estructura era muy parecida, y lo adapté al hotel: en vez de plazas en una clase, aquí se reserva una habitación entera para unas noches.

2. Análisis: historias de usuario

Antes de programar escribí las historias de usuario para tener claro qué tenía que poder hacer cada tipo de usuario. Hay tres tipos: el visitante anónimo, el cliente (que ha iniciado sesión) y el administrador del hotel. Cada historia tiene un criterio de aceptación en formato Given-When-Then (Dado-Cuando-Entonces).

HU-01 · Ver habitaciones disponibles

Como **visitante** quiero ver las habitaciones disponibles con su tipo, capacidad y precio por noche para decidir cuál me interesa.

Given que en el hotel hay 6 habitaciones y la 202 ya está reservada,
When entro en la página principal sin iniciar sesión,
Then veo las 5 habitaciones disponibles ordenadas por precio y la 202 no aparece.

HU-02 · Iniciar sesión

Como **cliente** quiero iniciar sesión con mi email y mi contraseña para poder reservar.

Given que estoy registrado como ana@iparhotel.com,
When escribo una contraseña incorrecta y pulso "Entrar",
Then no entro, veo el mensaje "Email o contraseña incorrectos" y la API responde 401.

HU-03 · Reservar una habitación

Como **cliente** quiero reservar una habitación indicando la fecha de entrada y las noches para asegurarme el alojamiento y saber cuánto me va a costar.

Given que he iniciado sesión y la Suite 302 (180 € la noche) está disponible,
When la reservo para dentro de una semana durante 6 noches,
Then se crea la reserva con un importe de 1.017,00 € ((180 × 6 + 50) − 10 %) y la habitación deja de estar disponible.

HU-04 · Avisar si la habitación ya no está libre

Como **cliente** quiero que la aplicación me avise si la habitación ya no está disponible para no pensar que tengo una reserva que no existe.

Given que la habitación 101 ya ha sido reservada por otro cliente,
When intento reservarla yo,
Then la API responde 409 Conflict con el mensaje "La habitación 101 no está disponible" y no se guarda nada.

HU-05 · Ver todas las reservas (administrador)
Como **administrador** quiero ver todas las reservas del hotel para saber qué habitaciones están ocupadas y cuánto se ha facturado.

Given que he iniciado sesión como administrador,
When entro en `/admin`,
Then veo todas las reservas con el cliente, las fechas y el importe, y el total al final. Si un cliente intenta entrar en `/admin` le sale "Acceso denegado".

3. Diseño: diagramas UML

3.1 Diagrama de casos de uso

En el diagrama salen los tres actores. El cliente puede hacer todo lo que hace el anónimo y además reservar, por eso hay una flecha de herencia (generalización) del cliente al anónimo, y lo mismo del administrador al cliente. Al reservar siempre se calcula el importe, así que lo puse como «include».

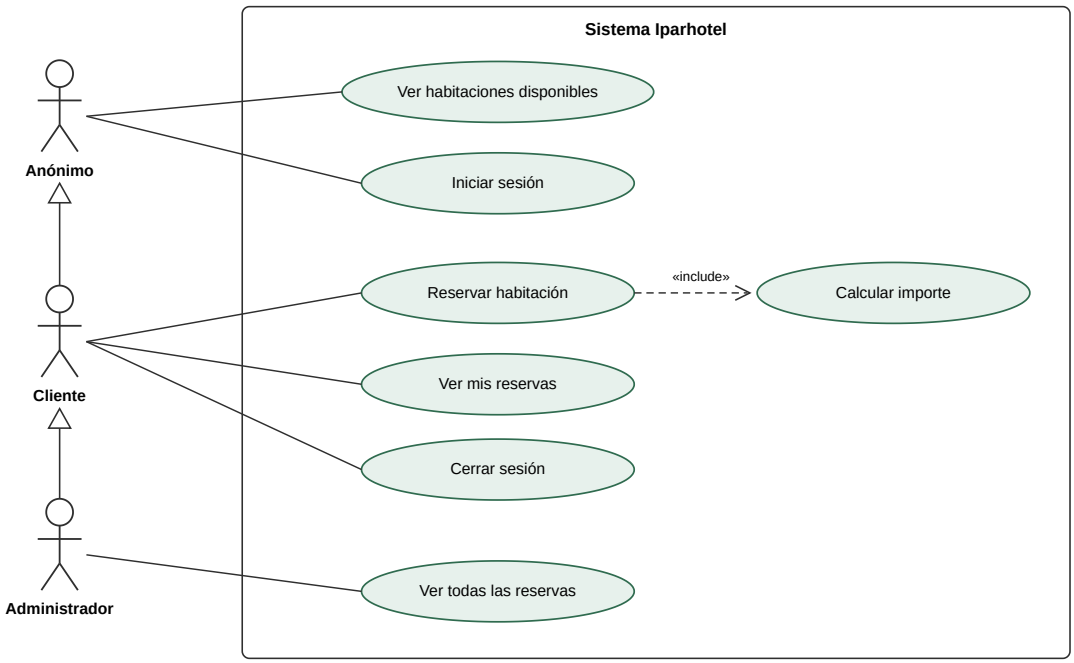


Figura 1. Diagrama de casos de uso

3.2 Diagrama de clases (modelo de la fase 1)

`Habitacion` es abstracta (está en cursiva) y tiene dos métodos abstractos que cada tipo de habitación implementa a su manera. `Suite` además implementa la interfaz `Promocionable` para el descuento. Una `Reserva` es de una habitación, y una habitación puede tener muchas reservas. Los errores de negocio se lanzan con `ReservaException`, que hereda de `RuntimeException`.

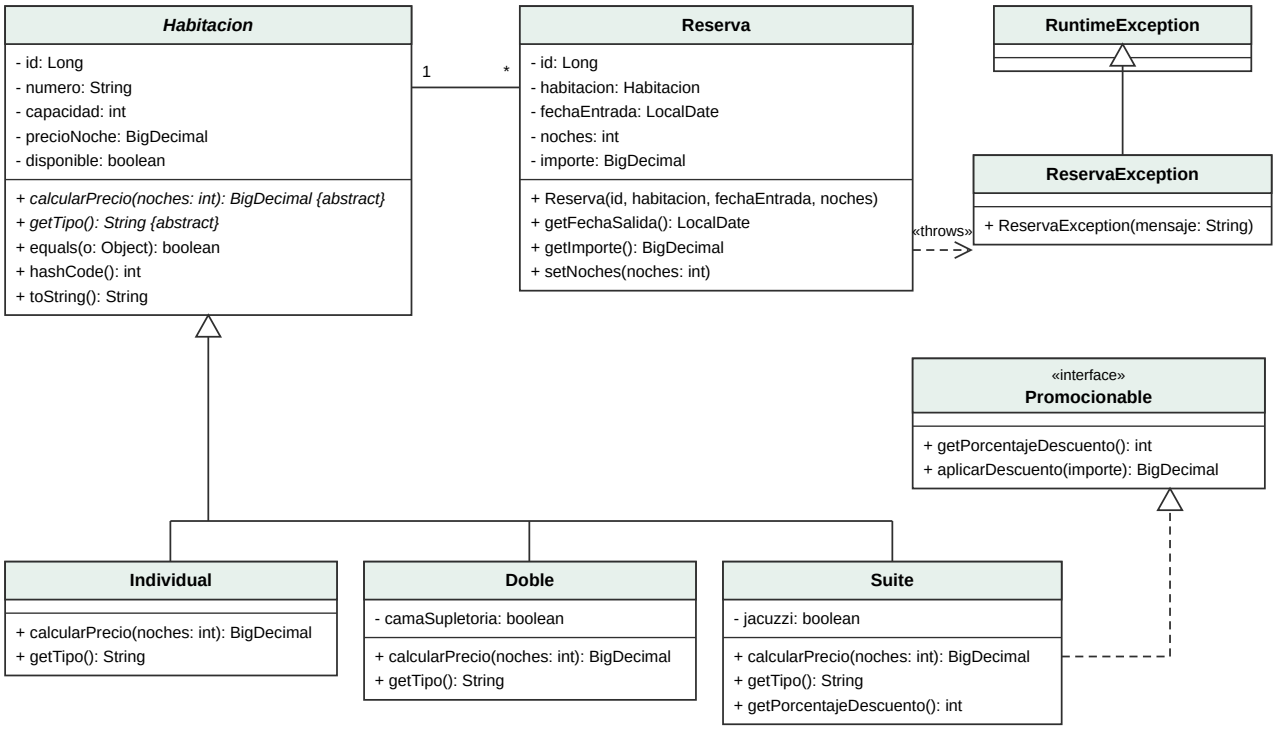


Figura 2. Diagrama de clases del modelo de la fase 1

4. Fase 1: modelo orientado a objetos

Qué había que hacer

Crear las clases del hotel en Java puro: una jerarquía de habitaciones con una clase abstracta, una interfaz para el descuento, la reserva y una excepción propia. Después probar el polimorfismo y los streams con un `main`.

Qué hice

- **Habitacion** (abstracta) con `id`, `numero`, `capacidad`, `precioNoche` (BigDecimal, porque con double los céntimos salen mal) y `disponible`. Tiene `equals` y `hashCode` por el id y un `toString`.
- **Individual**: precio = precioNoche × noches.
- **Doble**: lo mismo, más 20 € por noche si lleva cama supletoria.
- **Suite**: precioNoche × noches + 50 € de bienvenida, y a partir de 5 noches un 10 % de descuento con el método `default` de la interfaz `Promocionable`.
- **Reserva**: el importe no tiene setter ni se pasa en el constructor. Lo calcula siempre la propia reserva llamando a `habitacion.calcularPrecio(noches)`, así nadie puede poner un precio inventado.
- **ReservaException** hereda de `RuntimeException` y la lanzo, por ejemplo, si las noches son menos de 1.

En `PruebaPolimorfismo` recorro una `List<Habitacion>` y llamo a `calcularPrecio(3)` y `calcularPrecio(6)` sin usar `instanceof` ni `switch`: cada objeto sabe calcular su precio. Con streams saco las disponibles ordenadas por precio (`filter` + `sorted`) y sumo el importe de las reservas con `map` + `reduce`.

Terminal — java -cp target/classes com.iparhotel.PruebaPolimorfismo

```
=== Precio de 3 y 6 noches ===
Individual 101  3 noches:   180.00 EUR   6 noches:   360.00 EUR
Doble      201  3 noches:   270.00 EUR   6 noches:   540.00 EUR
Doble      202  3 noches:   345.00 EUR   6 noches:   690.00 EUR
Suite      301  3 noches:   500.00 EUR   6 noches:   855.00 EUR
Individual 102  3 noches:   165.00 EUR   6 noches:   330.00 EUR

=== Habitaciones disponibles por precio/noche ===
Individual 101  60 EUR/noche
Doble      201  90 EUR/noche
Suite      301  150 EUR/noche

=== Reservas ===
Reserva [id=1, habitacion=101 (Individual), fechaEntrada=2026-11-18, noches=3, importe=180.00]
Reserva [id=2, habitacion=202 (Doble), fechaEntrada=2026-11-21, noches=2, importe=230.00]
Reserva [id=3, habitacion=301 (Suite), fechaEntrada=2026-12-01, noches=6, importe=855.00]
Importe total de las reservas: 1265.00 EUR
```

Figura 3. Salida de la prueba de polimorfismo y streams. En la Suite 301 se ve el descuento: 6 noches = 855 € en vez de 950 €.

Problemas que tuve

- Al principio el símbolo € salía como "?" en la consola, así que lo cambié por "EUR".
- No tenía claro si el descuento de la suite se aplicaba antes o después de sumar los 50 € de bienvenida. Al final lo apliqué sobre el total.

5. Fase 2: aplicación con Spring Boot

Qué había que hacer

Convertir el proyecto en una aplicación Spring Boot con base de datos, una web hecha con Thymeleaf para ver habitaciones y reservar, y seguridad con Spring Security.

Qué hice

- **Persistencia:** añadí las dependencias (Web MVC, Data JPA, Validation, Thymeleaf, Security, Lombok y el driver de MySQL). Las clases de la fase 1 pasaron a ser entidades JPA. La herencia la mapeé con `@Inheritance(strategy = InheritanceType.SINGLE_TABLE)`: todas las habitaciones están en la tabla `habitaciones` y una columna `tipo_habitacion` dice si es INDIVIDUAL, DOBLE o SUITE.
- Creé **Usuario** (email único, contraseña codificada con BCrypt y rol CLIENTE o ADMINISTRADOR) y la reserva ahora tiene un usuario con `@ManyToOne`.
- **Datos de ejemplo** con un `CommandLineRunner`: 6 habitaciones (2 de cada tipo), un administrador y dos clientes, con las contraseñas ya codificadas.
- **Repositorios** con consultas derivadas: `findByDisponibleTrueOrderByPrecioNocheAsc()` y `findByUsuarioEmailOrderByFechaEntradaAsc(email)`.
- **ReservaService** con las dependencias por constructor y DTOs de entrada y salida. El método `reservar` comprueba que las noches estén entre 1 y 14, que la fecha sea futura y que la habitación exista y esté disponible. Si algo falla lanza `ReservaException`. Si todo va bien calcula el importe (polimorfismo otra vez) y marca la habitación como no disponible.
- **Vistas Thymeleaf** con una plantilla común (`layout.html`) para la cabecera, el menú y el pie.
- **Seguridad:** página de login propia, un `UserDetailsService` que busca el usuario en la base de datos, `/admin` solo para el administrador y el menú cambia según el usuario con `sec:authorize`.

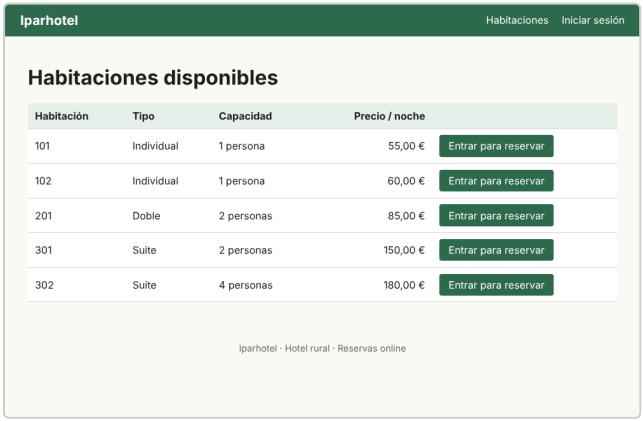


Figura 4. Página principal (pública)

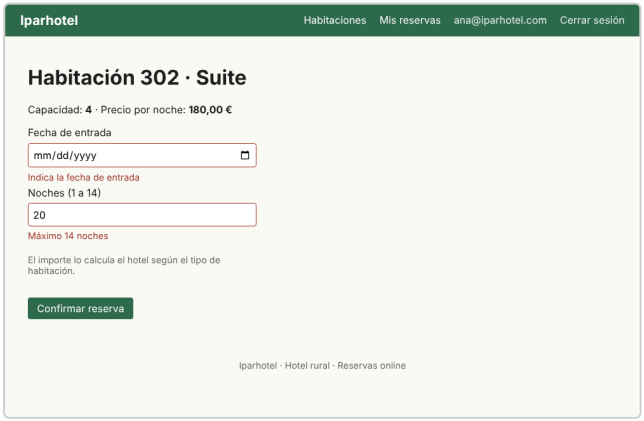


Figura 5. Formulario validado con @Valid y BindingResult

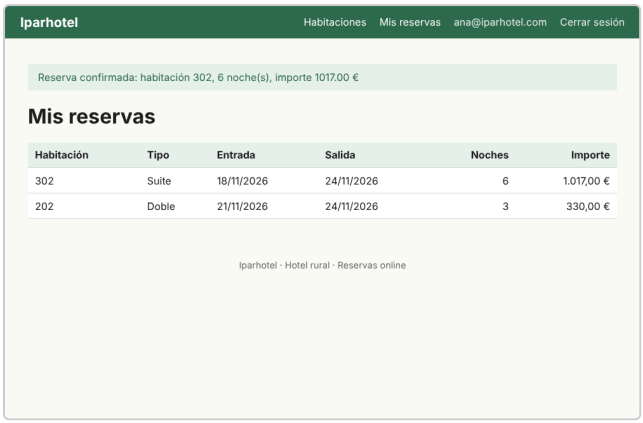


Figura 6. Tras reservar, redirección a "Mis reservas" con mensaje flash

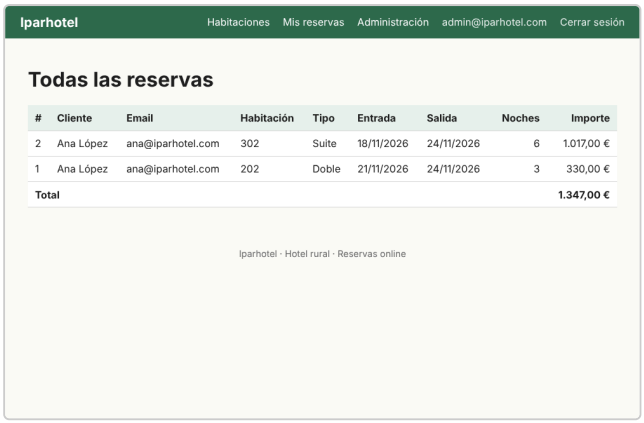


Figura 7. Vista del administrador con todas las reservas

Problemas que tuve

- Al tener dos clases con `main` (la de la fase 1 y la de Spring Boot), Maven no sabía cuál arrancar. Lo arreglé poniendo `start-class` en el `pom.xml`.
- En el `equals` usaba `otra.id` y con los objetos "proxy" de Hibernate el campo sale null. Lo cambié por `otra.getId()`.
- Al principio mostraba las reservas fuera de la transacción y daba `LazyInitializationException`. Lo solucioné convirtiendo a DTO dentro del servicio y con `@EntityGraph` en el repositorio.

6. Fase 3: API REST con JWT y Angular

6.1 API REST

Hice tres controladores con `@RestController` bajo `/api/v2` que usan el mismo `ReservaService` de la fase 2, así no tuve que repetir las reglas.

Método y ruta	Acceso	Respuesta
`GET /api/v2/habitaciones`	Público	Habitaciones disponibles
`POST /api/v2/usuarios/login`	Público	`{"accessToken": "...", "tokenType": "Bearer"}` o 401
`POST /api/v2/reservas`	Con token	201 Created, 400 datos no válidos, 404 no existe, 409 no disponible

El token es un JWT firmado con HS256. El *subject* es el email y lleva los claims `nombre` y `rol`. El secreto está en `application.properties` pero se lee de la variable de entorno `JWT_SECRETO`, para no subirlo a GitHub. En la seguridad configuré `oauth2ResourceServer` con JWT y CORS para `http://localhost:4200`. El usuario de la reserva se saca del token (`jwt.getSubject()`) y no del JSON, así un cliente no puede reservar a nombre de otro.

Como la web de la fase 2 sigue funcionando con su login de formulario, hice dos cadenas de seguridad: una para `/api/**` (sin sesión, con token) y otra para la web.

Terminal — pruebas de la API con curl

```
$ curl -s -X POST localhost:8080/api/v2/usuarios/login -H "Content-Type: application/json" -d
'{"email":"javier@iparhotel.com","password":"javier123"}'
{"accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJpcGFyaG9...", "tokenType": "Bearer"}

# payload del token (base64 decodificado)
{"iss":"iparhotel","sub":"javier@iparhotel.com","exp":1794395345,"iat":1794391745,"nombre":"Javi
er Etxeberria","rol":"CLIENTE"}

$ curl -i -X POST localhost:8080/api/v2/reservas -H "Authorization: Bearer $TOKEN" -d '{"idHabit
acion":1,"fechaEntrada":"2026-11-12","noches":3}'
HTTP/1.1 201
{"id":3,"numeroHabitacion":"101","tipoHabitacion":"Individual","nombreUsuario":"Javier Etxeberri
a","emailUsuario":"javier@iparhotel.com","fechaEntrada":"2026-11-12","fechaSalida":"2026-11-1
5","noches":3,"importe":165.00}

# la misma habitacion otra vez
HTTP/1.1 409
{"type":"about:blank","title":"Conflict","status":409,"detail":"La habitación 101 no está dispon
ible","instance":"/api/v2/reservas","motivo":"HABITACION_NO_DISPONIBLE"}

# noches = 20
HTTP/1.1 400
{"type":"about:blank","title":"Bad Request","status":400,"detail":"Los datos enviados no son vál
idos","instance":"/api/v2/reservas","errores":{"noches":"Máximo 14 noches"}}

# sin token
HTTP/1.1 401
{"type":"about:blank","title":"Unauthorized","status":401,"detail":"Necesitas un token válido pa
ra acceder a este recurso."}
```

Figura 8. Llamadas a la API: login, contenido del token, 201, 409, 400 y 401

6.2 Aplicación Angular

Creé el proyecto con Angular CLI (`ng new iparhotel-web`) y los componentes, servicios y el guard con `ng generate`. Tiene tres rutas:

- `''` (principal): lista las habitaciones con `@for`. Cada fila es un componente hijo, `habitacion-fila`, que recibe la habitación con `input()` y avisa al padre con `output()` cuando se pulsa "Reservar".
- `login`: al entrar guarda el token en `JwtService`, que tiene dos signals calculados (`autenticado` y `nombre`), y vuelve a la principal. La cabecera lee esos signals y muestra "Hola, nombre" y el botón de cerrar sesión.
- `reservar/:id`: protegida con un guard que manda a `/login` si no hay sesión. Tiene un formulario de fecha y noches que llama a la API con la cabecera `Authorization: Bearer` y muestra el importe que calcula el servidor o el error que devuelve.

Para cargar los datos al abrir una página usé el método del ciclo de vida `ngOnInit()`.

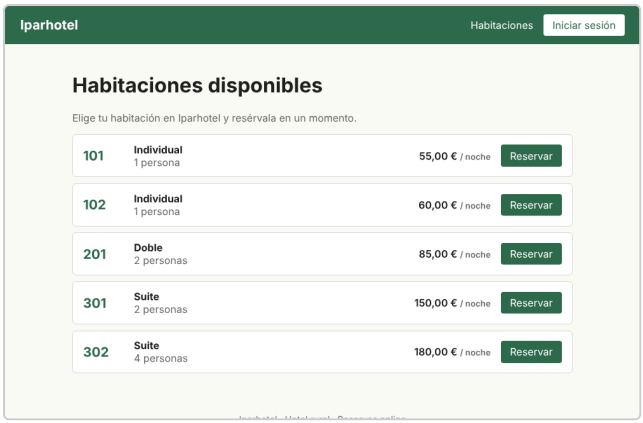


Figura 9. Principal sin sesión

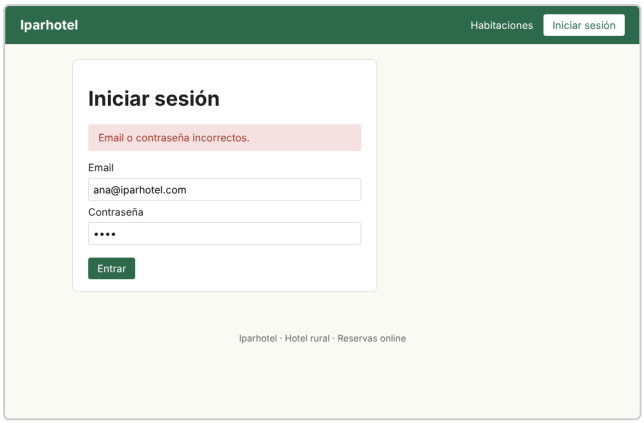


Figura 10. Login con contraseña incorrecta (401)

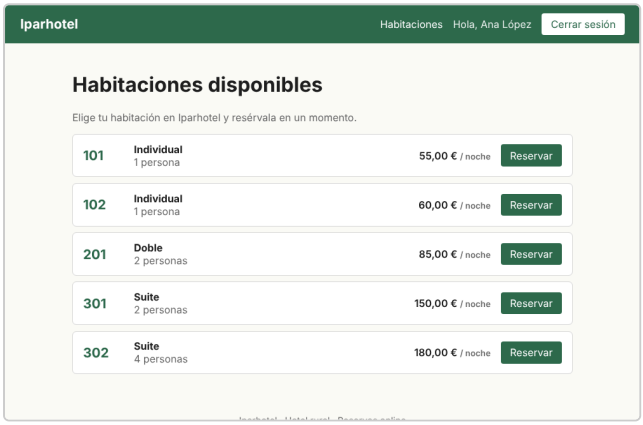


Figura 11. Con sesión: la cabecera muestra el nombre

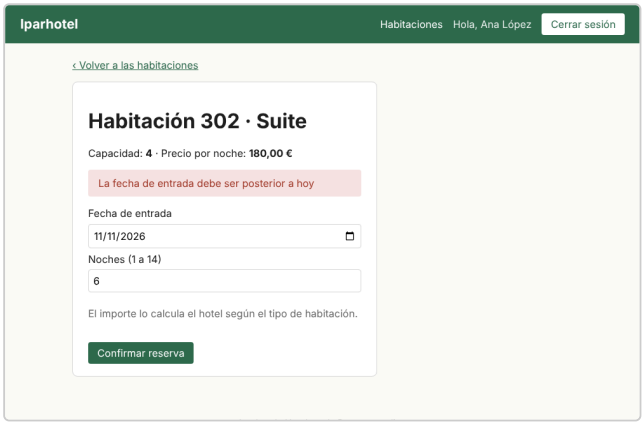


Figura 12. Error que devuelve la API (fecha de hoy)

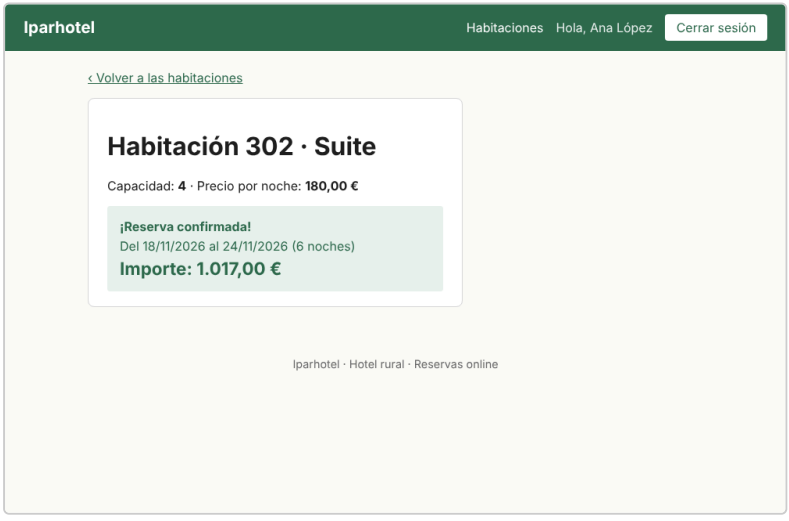


Figura 13. Reserva confirmada con el importe calculado por la API

Problemas que tuve

- Sin la variable `JWT_SECRETO` la aplicación no arranca. Al principio pensé que era un error, pero es a propósito: así no se puede arrancar con un secreto por defecto.
- El `@ControllerAdvice` de la web también cogía los errores de la API y los redirigía a una página HTML. Lo limité con `basePackages` para que cada uno tenga su manejador de errores.
- Al llamar desde Angular (puerto 4200) a la API (puerto 8080) el navegador bloqueaba la petición por CORS hasta que lo configuré en Spring.

7. Pruebas

Fui haciendo pruebas en cada fase. En Java uso JUnit 5, Mockito y MockMvc; las pruebas usan una base de datos H2 en memoria para no necesitar MySQL. En Angular uso las pruebas que trae Angular CLI (Vitest).

Clase de prueba	Qué comprueba	Pruebas
HabitacionTest	Precios de cada tipo, descuento de la suite, equals/hashCode	11
ReservaTest	Importe calculado por el sistema y ReservaException	5
ReservaServiceTest	Reglas de reservar() con Mockito (noches, fecha, no disponible...)	6
RepositoriosTest	Consultas derivadas y SINGLE_TABLE con @DataJpaTest	2
WebSeguridadTest	Login, /admin solo para administrador, formulario y mensaje flash	8
ApiTest	Token JWT con sus claims, 201, 400, 401, 404, 409 y CORS	7
Angular (3 ficheros)	JwtService, guard y componente principal	7

En total son **39 pruebas en Java y 7 en Angular**, y todas pasan (`mvn test` y `ng test`).

8. Cómo arrancar el proyecto

1. Tener MySQL en marcha (la base de datos `iparhotel` se crea sola). Usuario y contraseña por defecto `root` / `root`, se cambian con las variables `DB_USER` y `DB_PASSWORD`.
2. Definir el secreto del token (32 caracteres o más): `JWT_SECRETO=un-secreto-largo-de-al-menos-32-caracteres`
3. En la carpeta `iparhotel`: `mvn spring-boot:run`. La web está en http://localhost:8080.
4. En la carpeta `iparhotel-web`: `npm install` y `ng serve`. Angular está en http://localhost:4200.

Usuario	Contraseña	Rol
admin@iparhotel.com	admin123	ADMINISTRADOR
ana@iparhotel.com	ana123	CLIENTE
javier@iparhotel.com	javier123	CLIENTE

9. Conclusiones

Lo que más me ha gustado es ver cómo el mismo modelo de la fase 1 se ha ido usando en todo el proyecto: el método `calcularPrecio` que hice al principio es el que calcula el importe que sale en la web y en Angular. También he entendido mejor la diferencia entre la seguridad con sesión (web con Thymeleaf) y con token (API con JWT), porque al final he tenido que tener las dos a la vez.

Lo que más me costó fue la parte de seguridad y los errores de CORS. Si tuviera más tiempo añadiría poder cancelar reservas, comprobar que no se solapen las fechas (ahora la habitación se marca como no disponible directamente) y una página de "Mis reservas" en Angular.